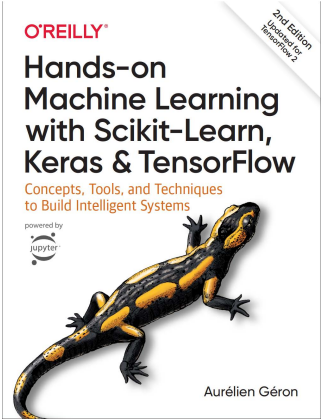
CITS5508 Machine Learning
Regression Models

Marcell Szikszai

2026

THE UNIVERSITY OF
WESTERN
AUSTRALIA

Reading



Chapter 4 – Training Models

Hands-on Machine Learning with Scikit-Learn & TensorFlow,
Aurélien Géron,
O'Reilly Media, 2022.

Available online through UWA OneSearch.

Example code and solutions to the exercises:
`handson-ml3/04_training_linear_models.ipynb`

Lecture 3

Today, we'll start to look at how Machine Learning algorithms work, starting with some regression models.

We will cover:

- Linear Regression
- Gradient Descent
- Polynomial Regression
- Learning Curves
- Regularised Linear Models
- Logistic Regression

Linear Regression

This is a linear model which makes a prediction by simply computing a weighted sum of the input features, plus a constant called the *bias* term (also called the *intercept* term),

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots + \theta_n x_n,$$

where

- $\hat{y}$ is the predicted value;
- n is the number of features;
- x_i is the ith feature value; and
- θ_j is the jth model parameter.

Linear Regression

...Or in (column) vectors ...

$$\hat{y} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta}^\top \mathbf{x},$$

where

- $\hat{y}$ is the predicted value;
- $\boldsymbol{\theta}$ is the model's parameter vector, containing the bias term θ_0 and the feature weights $\boldsymbol{\theta} = (\theta_0, \theta_1, \cdots, \theta_n)^\top$;
- $\mathbf{x}$ is the instance's feature vector, $\mathbf{x} = (1, x_1, \cdots, x_n)^\top$;
- $\boldsymbol{\theta}^\top \mathbf{x}$ is the matrix multiplication of $\boldsymbol{\theta}^\top$ and $\mathbf{x}$; and
- $h_{\boldsymbol{\theta}}$ is the hypothesis function, using the model parameters $\boldsymbol{\theta}$.

Linear Regression – training the model

- Training a model means setting its parameters so that the model best fits the training set.
- We first need a measure of how well (or poorly) the model fits the training data.

- Therefore, to train a Linear Regression model, you need to find the value of $\boldsymbol{\theta}$ that minimises some cost function $C(\boldsymbol{\theta})$,

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} C(\boldsymbol{\theta}).$$

Linear Regression – training the model

Let's start by picking a performance measure (or cost function).

A common performance measure of a regression model is the *Mean Squared Error* (*MSE*):

$$\mathrm{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}}) = \frac{1}{m} \sum_{i=1}^{m} \left(\boldsymbol{\theta}^\top \mathbf{x}^{(i)} - y^{(i)} \right)^2 .$$

Or the *Residual Sum of Squares* (*RSS*):

$$\mathrm{RSS}(\mathbf{X}, h_{\boldsymbol{\theta}}) = \sum_{i=1}^{m} \left(\boldsymbol{\theta}^\top \mathbf{x}^{(i)} - y^{(i)} \right)^2 ,$$

Linear Regression – minimising RSS

"To train a Linear Regression model, we need to find the value of $\boldsymbol{\theta}$ that minimises some cost function."

$$\begin{aligned}\text{RSS}(\mathbf{X}, h_{\boldsymbol{\theta}}) &= \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|^2, \\ &= (\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\theta}).\end{aligned}$$

Differentiate with respect to $\boldsymbol{\theta}$:

$$\frac{\partial \text{RSS}}{\partial \boldsymbol{\theta}} = -2\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\boldsymbol{\theta}).$$

Set derivative to zero to find the minimum:

$$\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) = 0,$$

Finally, solve for $\boldsymbol{\theta}$:

$$\hat{\boldsymbol{\theta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

The Normal Equation

To find the value of $\boldsymbol{\theta}$ that minimizes the cost function, there is a closed-form solution—in other words, a mathematical equation that gives the result directly. This is called the **Normal Equation**.

$$\hat{\boldsymbol{\theta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

where

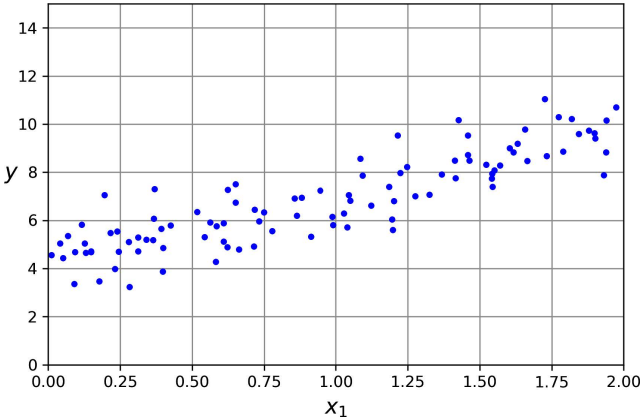
- $\mathbf{X}$ is a $m \times (n+1)$ matrix – the ith row of $\mathbf{X}$ is the training instance $\mathbf{x}^{(i)}$;
- $\hat{\boldsymbol{\theta}}$ is the value that minimizes the cost function;
- $\mathbf{y}$ is the vector of target values containing $y^{(1)}$ to $y^{(m)}$.

Once the optimal model parameter $\hat{\boldsymbol{\theta}}$ is known, given a new test instance $\mathbf{x}_{\text{test}}$, we can predict y_{test} using the formula:

$$y_{\text{test}} = \hat{\boldsymbol{\theta}}^\top \mathbf{x}_{\text{test}}.$$

A random dataset to use as an example

```
import numpy as np
X = 2 * np.random.rand(100, 1)
y = 4 + 3 * X + np.random.randn(100, 1)
```



Example: Fitting the model

Now let's compute $\hat{\boldsymbol{\theta}}$ using the Normal Equation.

```
>>> X_b = np.c_[np.ones((100, 1)), X] # add x0 = 1 to each instance
>>> theta_best = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y
>>> theta_best
array([[4.21509616],
       [2.77011339]])
```

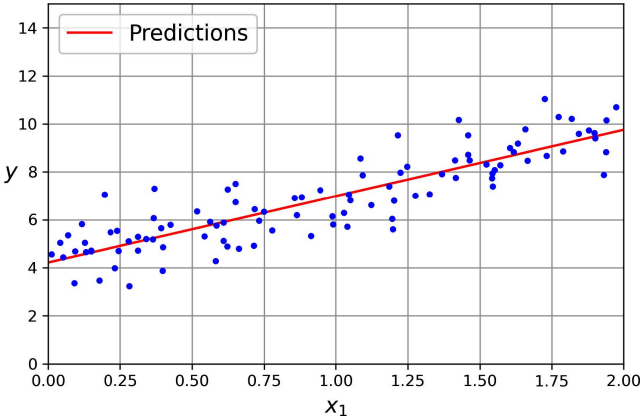
Making Predictions

```
>>> X_new = np.array([[0], [2]])
>>> X_new_b = np.c_[np.ones((2, 1)), X_new]  # add x0 = 1 to each instance
>>> y_pred = X_new_b @ theta_best
>>> y_pred
array([[ 4.21509616],
       [ 9.75532293]])
```

Let's plot this model's predictions:

```
>>> plt.plot(X_new, y_predict, "r-")
>>> plt.plot(X, y, "b.")
>>> plt.axis([0, 2, 0, 15])
>>> plt.show()
```

Plot of Predictions



Linear Regression with Scikit-Learn

The equivalent code using Scikit-Learn looks like this:

```
>>> from sklearn.linear_model import LinearRegression
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([ 4.21509616]), array([[ 2.77011339]]))
>>> lin_reg.predict(X_new)
array([[ 4.21509616],
       [ 9.75532293]])
```

Beware: running time of normal equation can be cubic in number of features but linear with respect to the number of instances in the training set. Predictions are fast.

Gradient Descent

Gradient Descent is a very generic optimization algorithm for which the general idea is to tweak parameters iteratively in order to minimize a cost function.

It measures the local gradient of the error function with regards to the parameter vector $\boldsymbol{\theta}$, and it goes in the direction of descending gradient. Once the gradient is zero, you have reached a minimum!

It starts by filling $\boldsymbol{\theta}$ with random values (this is called random initialization), and then improve it gradually, taking one step at a time, each step attempting to decrease the cost function (e.g., the MSE), until the algorithm converges to a minimum.

Partial Derivatives and the Gradient Vector

To implement Gradient Descent, we need to compute the gradient of the cost function with regards to each model parameter θ_j.

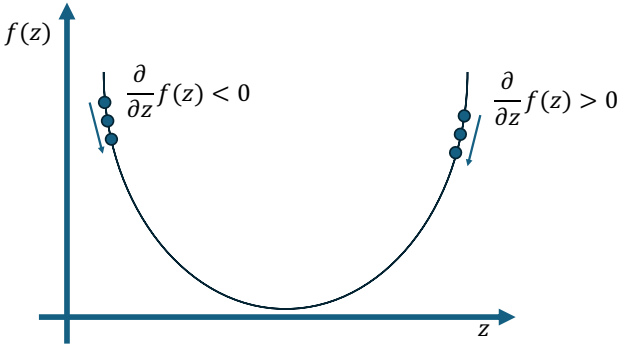
That is, we need to calculate how much the cost function will change if you change θ_j just a little bit. This is called a *partial derivative*.

$$\frac{\partial}{\partial \theta_j} \mathrm{MSE}(\boldsymbol{\theta}) = \frac{2}{m} \sum_{i=1}^{m} \left(\boldsymbol{\theta}^\top \mathbf{x}^{(i)} - y^{(i)} \right) x_j^{(i)}$$

Partial Derivatives and the Gradient Vector

The idea:

- when the $\frac{\partial}{\partial z} f(z)$ is negative at $z^{(t)}$, then $z^{(t+1)}$ need to be $z^{(t)}$ + "something" to go in the direction of the minimum of $f(z)$;
- when the $\frac{\partial}{\partial z} f(z)$ is positive at $z^{(t)}$, then $z^{(t+1)}$ need to be $z^{(t)}$ − "something" to go in the direction of the minimum of $f(z)$;



Partial Derivatives and the Gradient Vector

The gradient vector contains all the partial derivatives of the cost function:

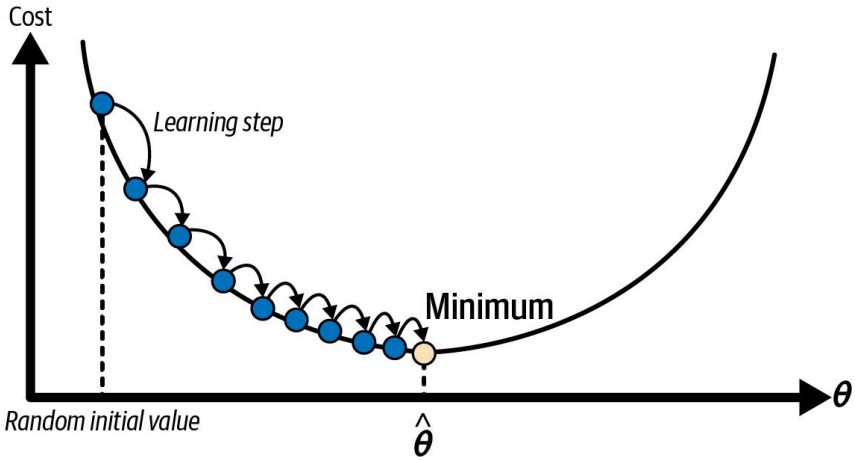
$$\mathrm{grad}(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} \mathrm{MSE}(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} \mathrm{MSE}(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_1} \mathrm{MSE}(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial}{\partial \theta_n} \mathrm{MSE}(\boldsymbol{\theta}) \end{pmatrix} = \frac{2}{m} \mathbf{X}^\top (\mathbf{X}\boldsymbol{\theta} - \mathbf{y}).$$

Then, the Gradient Descent step is:

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \, \mathrm{grad}(\boldsymbol{\theta}^{(t)}),$$

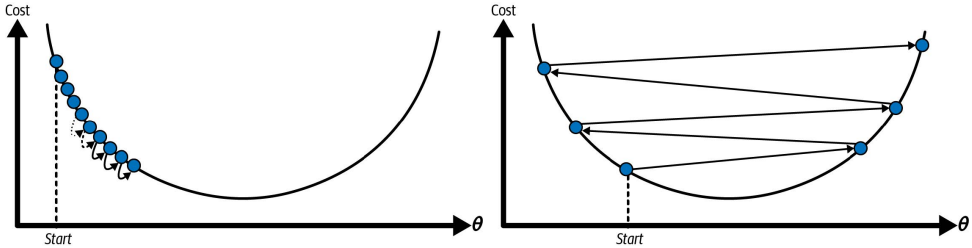
where $\boldsymbol{\theta}^{(t)}$ is the value of $\boldsymbol{\theta}$ at the previous step, $\boldsymbol{\theta}^{(t+1)}$ is the new value of $\boldsymbol{\theta}$ (next step), and η is the *learning rate* (step size).

Gradient Descent

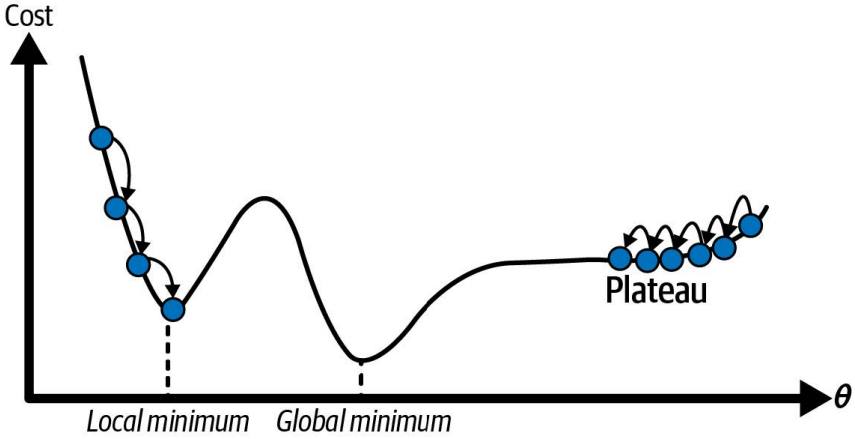


Learning Rate

An important parameter in Gradient Descent is the size of the steps, determined by the *learning rate* hyperparameter (η).

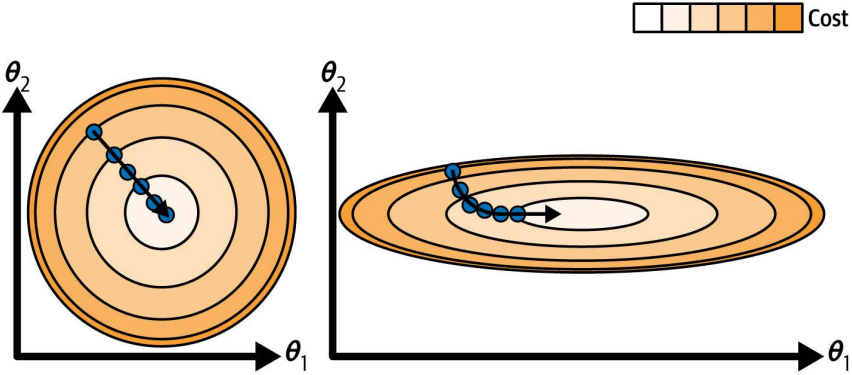


Gradient Descent – pitfalls



Gradient Descent for a Regression with an MSE cost function

Will be a bowl shape (good) but may be elongated if there are different scales.

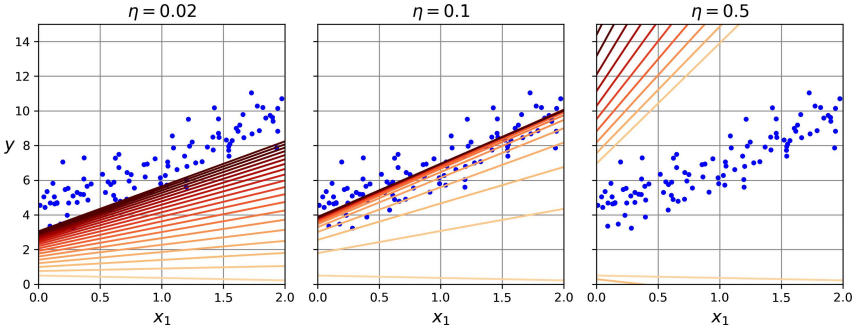


GD – implementation

```
eta = 0.1 # learning rate
n_epochs = 1000
m = len(X_b) # number of instances

np.random.seed(42)
theta = np.random.randn(2, 1) # randomly initialized model parameters

for epoch in range(n_epochs):
  gradients = 2 / m * X_b.T @ (X_b @ theta - y)
  theta = theta - eta * gradients
```

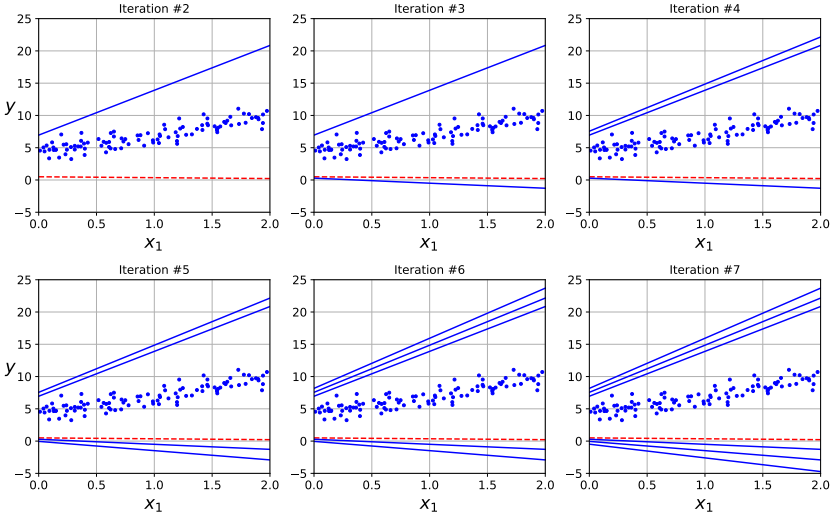
```
>>> theta
array([[4.21509616],
       [2.77011339]])
```

GD with various learning rates



GD with various learning rates

A divergence example -- when the learning rate is too high ($\eta = 0.5$ in this plot)



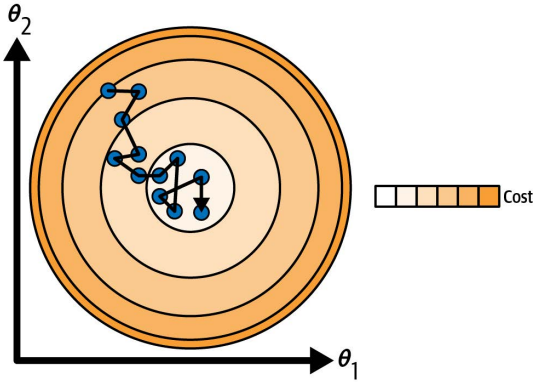
3 Ways to do Gradient Descent

Batch GD: Uses the whole training set to compute the gradients at every step, which makes it very slow when the training set is large.

Stochastic GD: Just picks a random instance in the training set at every step and computes the gradients based only on that single instance. Makes the algorithm much faster since it has very little data to manipulate at every iteration. Also possible to train on huge training sets.

Mini-batch GD: At each step, computes the gradients on small random sets of instances (*mini-batches*). Get a performance boost from hardware optimization of matrix operations (e.g., vectorisation, GPUs).

Stochastic Gradient Descent (SGD) jumps around

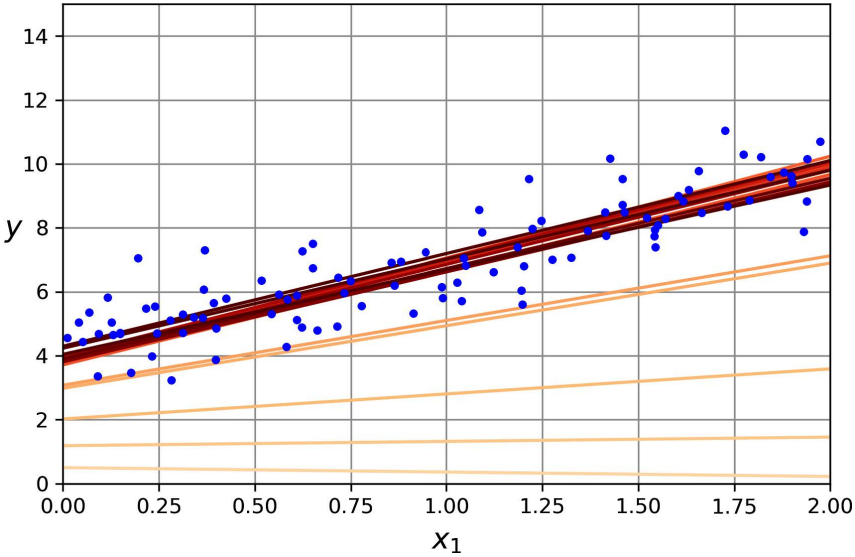


One option is to gradually reduce the learning rate to allow the algorithm to settle to the global minimum.

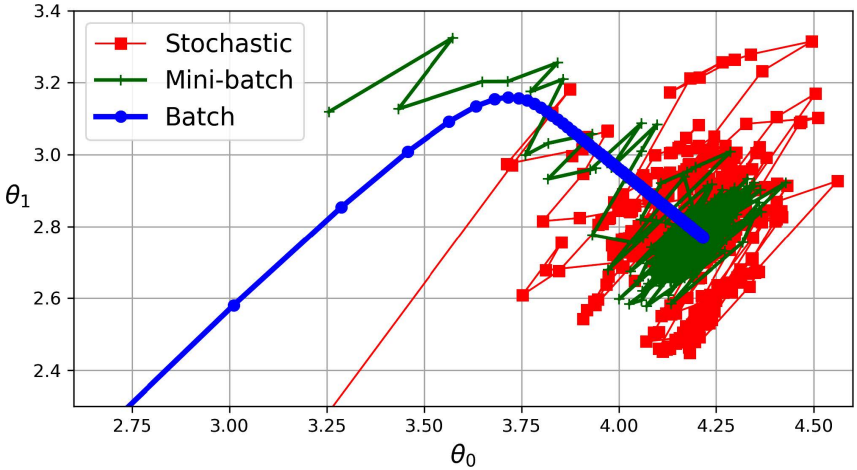
Stochastic Descent – reducing the learning rate gradually

```
n_epochs = 50
t0, t1 = 5, 50 # learning schedule hyperparameters

def learning_schedule(t):
  return t0 / (t + t1)

np.random.seed(42)
theta = np.random.randn(2, 1) # random initialization

for epoch in range(n_epochs):
  for iteration in range(m):
      random_index = np.random.randint(m)
      xi = X_b[random_index : random_index + 1]
      yi = y[random_index : random_index + 1]
      gradients = 2 * xi.T @ (xi @ theta - yi)
      eta = learning_schedule(epoch * m + iteration)
      theta = theta - eta * gradients
```

Stochastic Descent in Action



Paths in Parameter Space



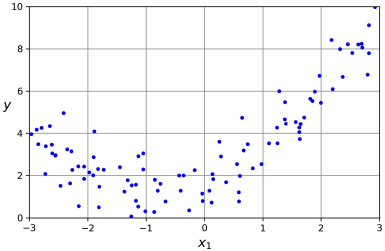
Comparing approaches for training Linear Regression

Algorithm	Large m	Large n	Hyperparams	Scaling	Scikit-Learn
Normal equation	Fast	Slow	0	No	N/A
SVD	Fast	Slow	0	No	`LinearRegression`
Batch GD	Slow	Fast	2	Yes	N/A
Stochastic GD	Fast	Fast	$\geq$2	Yes	`SGDRegressor`
Mini-batch GD	Fast	Fast	$\geq$2	Yes	N/A

- *Singular Value Decomposition (SVD)* is a technique used for solving the normal equations in a numerically stable way.
- *Hyperparameters*: learning rate η, number of iterations, batch size, momentum etc.

Polynomial Regression

What if your data is actually more complex than a straight line?




```
np.random.seed(42)
m = 100
X = 6 * np.random.rand(m, 1) - 3
y = 0.5 * X ** 2 + X + 2 +
    np.random.randn(m, 1)
```

In the above example, $y = 0.5x_1^2 + x_1 + 2 + \epsilon$, where $\epsilon \sim \mathcal{N}(0, 1)$.

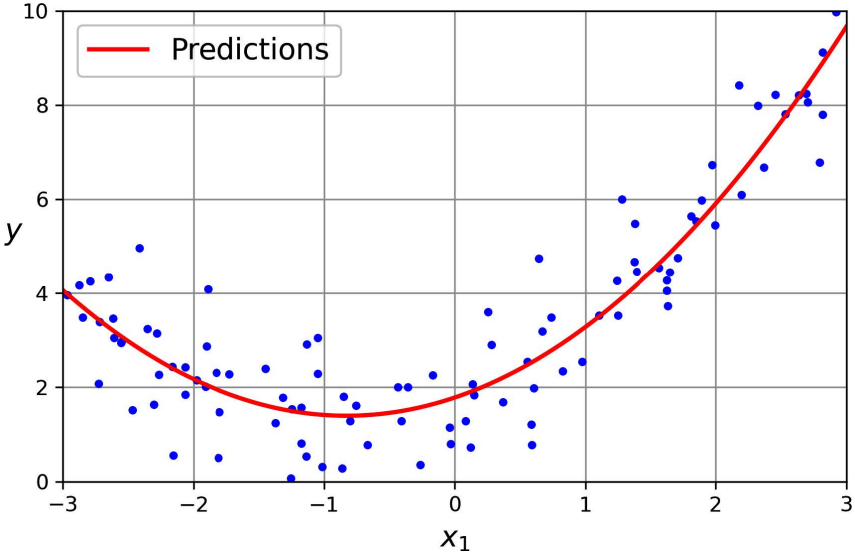
Polynomial Regression

We can still use linear regression, but we introduce a new feature which is x_1^2.

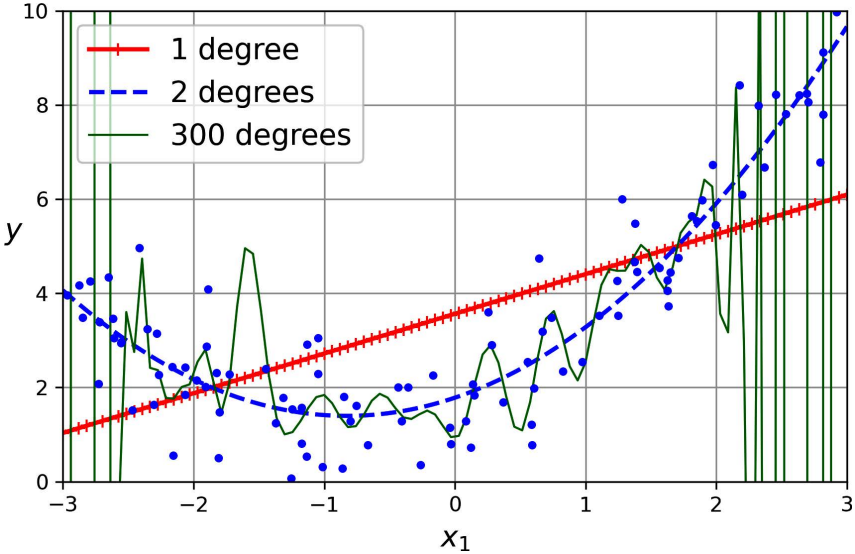
```
>>> from sklearn.preprocessing import PolynomialFeatures
>>> poly_features =
        PolynomialFeatures(degree=2, include_bias=False)
>>> X_poly = poly_features.fit_transform(X)
>>> X[0]
array([-0.75275929])
>>> X_poly[0]
array([-0.75275929, 0.56664654])
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X_poly, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([1.781]), array([[0.933, 0.564]]))
```

The fitted curve (red) is: $\hat{y} = 0.56x_1^2 + 0.93x_1 + 1.78 = 0$.

Polynomial Regression



Polynomial Regression – overfitting



Polynomial Regression – overfitting

Of course, this high-degree Polynomial Regression model is severely overfitting the training data, while the linear model is underfitting it.

The model that will generalize best in this case is the quadratic model. It makes sense since the data was generated using a quadratic model, but in general you won't know what function generated the data, so how can you decide how complex your model should be?

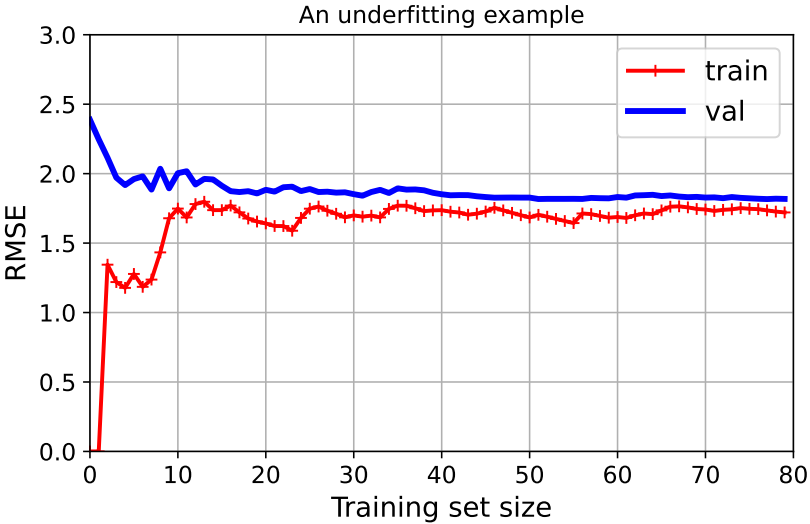
How can you tell that your model is overfitting or underfitting the data?

Learning Curves

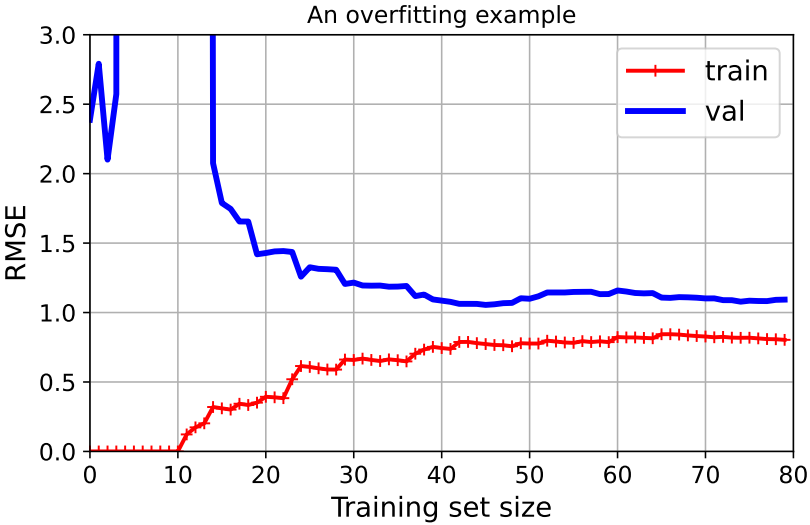
Learning Curves are plots of the model's performance on the training set and the validation set as a function of the **training set size**. To generate the plots, train the model several times on different sized subsets of the training set.

See the code in the text book for `plot_learning_curves(model, X, y)`.

Learning Curves – underfitting



Learning Curves – overfitting



Logistic Regression

Some regression algorithms can be used for classification as well (and vice versa).

Logistic Regression (also called *Logit Regression*) is commonly used to estimate the probability that an instance belongs to a particular class (e.g., what is the probability that this email is spam?).

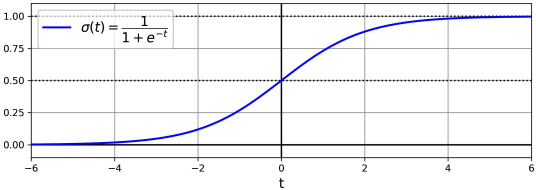
How does a logistic model make a prediction?

Just like a *Linear Regression* model, a *Logistic Regression* model computes a weighted sum of the input features (plus a bias term).
The probability $\hat{p}$ estimated by the Logistic Regression model is given by:

$$\hat{p} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \sigma(\boldsymbol{\theta}^\top \mathbf{x})$$

where the logistic function $\sigma(t)$ is defined as follows:

$$\sigma(t) = \frac{1}{1 + \exp(-t)}$$



How does a logistic model make a prediction?

The probability $\hat{p}$ estimated by the Logistic Regression model is given by:

$$\hat{p} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \sigma(\boldsymbol{\theta}^\top \mathbf{x}) = \frac{1}{1 + \exp(-\boldsymbol{\theta}^\top \mathbf{x})}$$

The Logistic Regression model prediction is:

$$\hat{y} = \begin{cases} 0 & \text{if } \hat{p} < 0.5 \\ 1 & \text{if } \hat{p} \geq 0.5 \end{cases}$$

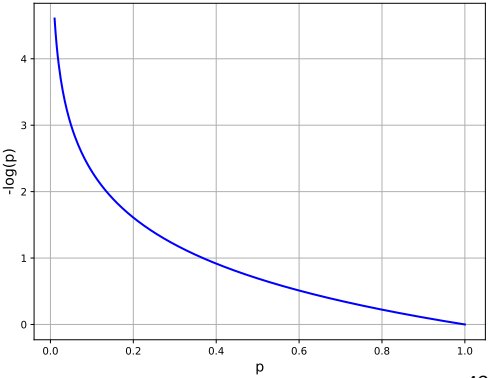
A Logistic Regression model predicts 1 if $\boldsymbol{\theta}^\top \mathbf{x}$ is positive, and 0 if it is negative.

Training and Cost Function

We want to find the parameter vector $\boldsymbol{\theta}$ so that the model gives high probabilities for instances in the positive class and low probabilities for instances in the negative class.

Cost function of a single instance:

$$C(\boldsymbol{\theta}) = \begin{cases} -log(\hat{p}) & \text{if } y = 1 \\ -log(1-\hat{p}) & \text{if } y = 0 \end{cases}$$



Therefore, $\hat{p} \in [0, 1]$.

Training and Cost Function

$$C(\boldsymbol{\theta}) = \begin{cases} -log(\hat{p}) & \text{if } y = 1 \\ -log(1-\hat{p}) & \text{if } y = 0 \end{cases}$$

Example 1: $\hat{p} = 0.1$:
If $y = 1$: $C(\boldsymbol{\theta}) = -log(0.1) = 2.30$
If $y = 0$: $C(\boldsymbol{\theta}) = -log(1 - 0.1) = 0.10$

Example 2: $\hat{p} = 0.8$:
If $y = 1$: $C(\boldsymbol{\theta}) = -log(0.8) = 0.22$
If $y = 0$: $C(\boldsymbol{\theta}) = -log(1 - 0.8) = 1.6$

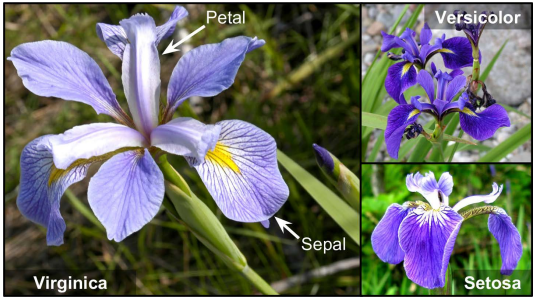
Training and Cost Function

The *Logistic Regression cost function* (*log loss*) is:

$$J(\boldsymbol{\theta}) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(\hat{p}^{(i)}) + (1-y^{(i)})\log(1-\hat{p}^{(i)})\right]$$

- We want that the model estimates high probabilities for positive instances ($y = 1$) and low probabilities for negative instances.
- Can do this using a simple log function of the probability.
- Average over all instances.
- No known closed-form equation to compute the value of $\boldsymbol{\theta}$ that minimizes this cost function.
- But cost function is convex, so GD is guaranteed to find the global minimum (if the learning rate is not too large and you wait long enough).

Iris Example: three species



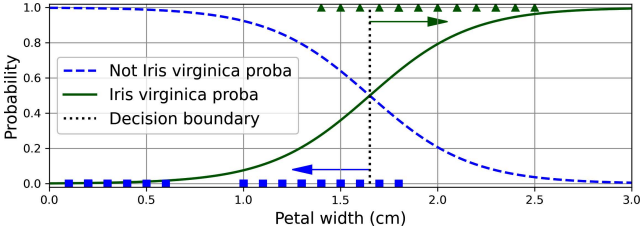
Some flower properties vary with the species.

A 3-class classification problem, with 4 attributes (all in cm): *sepal length*, *sepal width*, *petal length*, and *petal width*.

Just using petal length feature

Suppose that we want to do a binary classification: to classify *Iris-Virginica* type versus *not Iris-Virginica* type, using just the *petal width* feature.

After training a Logistic Regression classifier, we get a *decision boundary* at about 1.6cm.

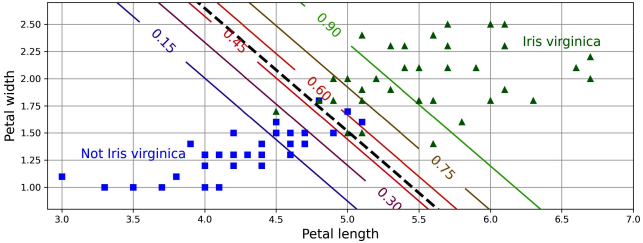


green triangles: Iris-Virginica; blue squares: other iris species

With two features...

Again, we use the Logistic Regression classifier but on two features: *petal length* and *petal width*.

... we also get a linear boundary.



The black dashed line represents the model's decision boundary (50% probability)

And that's all for the third lecture.

Have a good week.